WORKBOOK: WORKED EXAMPLES AND TRACING
PrepWise Synthetic Demo Material - Systematic State Tracing Guide
==========

2.1 The Purpose of Trace Tables

When analyzing complex control flow, tracing state variable-by-variable prevents logic errors and uncovers off-by-one boundary failures.

2.2 Methodology for Constructing a Trace Table

1. List all state variables and loop guards as column headers.
2. Record row 0 as the initial state before loop entry.
3. For each iteration, evaluate the loop guard condition.
4. Record new values in the row only when an assignment occurs.
5. Continue until the loop guard evaluates to False.

PrepWise Synthetic Demo Material - Worked Example 1: While-Loop Trace
==========

Given the following code snippet:

```
total = 0
num = 2
while num <= 6:
    total += num
    num += 2
```

Trace Table Analysis:

Step	num	num <= 6?	total (before)	total += num	num += 2
Init	2	(entry)	0	-	-
Iter1	2	True	0	2	4
Iter2	4	True	2	6	6
Iter3	6	True	6	12	8
Exit	8	False	12	-	-

Final Values at Program Termination:
total = 12, num = 8 (Note that num reaches 8 to falsify the condition!).

==========

Pitfall 1: Off-By-One Bound in range()
Bug: range(1, 5) iterates 4 times with values 1, 2, 3, 4.
If you need 1 through 5 inclusive, use range(1, 6).

Pitfall 2: Re-initializing the Accumulator Inside the Loop
Bug: Putting total = 0 inside the while block resets the accumulator
on every iteration, leaving only the final element's value.

Pitfall 3: Inadvertent Infinite Loop
Bug: Incrementing the wrong variable or placing counter increment
inside an if block that fails to trigger.

```
    while i < 10:
        if i % 2 == 0:
            i += 1  # If i is odd, it becomes an infinite loop!
```

WORKBOOK: WORKED EXAMPLES AND TRACING
PrepWise Synthetic Demo Material - Practice Exercises with Rubrics
==========

Exercise A: Factorial Product Accumulator
Problem: Write a loop computing n! for n = 5.
Rubric:
- Initialized product to 1 (not 0): 2 pts
- Correct loop guard (counter <= n or counter > 0): 2 pts
- Monotonic decrement/increment of counter: 1 pt

Exercise B: Identify the Termination Bug
Snippet:

```
k = 10
while k > 0:
    print(k)
    # Missing: k -= 1
```

Diagnostic: k is never modified; condition k > 0 is perpetually True.
Correction: Insert k -= 1 inside the loop body.